Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]

Wentao Huang
National University of Singapore
huangwentao@u.nus.edu

Qiming Huang
National University of Singapore
huang_qiming@u.nus.edu

Yunhong Ji
Renmin University of China
jiyunhong@ruc.edu.cn

Mian Lu
4Paradigm Inc.
lumian@4paradigm.com

Yuqiang Chen
4Paradigm Inc.
chenyuqiang@4paradigm.com

Bingsheng He
National University of Singapore
dcsheb@nus.edu.sg

Kian-Lee Tan
National University of Singapore
dcstankl@nus.edu.sg

ABSTRACT

Sparse general matrix-matrix multiplication (SpGEMM) is a key kernel in scientific computing, graph analytics, and relational query processing. Yet, its performance remains a significant challenge, due to the inherent sparsity and, more critically, the prevalent skew patterns often found in real-world workloads. This paper looks into this problem and demonstrates that an unbalanced workload distribution is the main culprit, harming performance through two co-existing root causes: first, *multi-dimensional skew*, where imbalance across computation cost, output density, and data locality defies any single balancing metric; and second, *cache contention*, where dense substructures induce severe on-chip interference.

To that end, we propose adaptive morsel scheduling (AMS), a dynamic scheduling framework that integrates contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact) to alleviate cache-level contention, alongside an hybrid accumulation mechanism (HYB) that adaptively selects the best accumulator per morsel. Evaluations across SpGEMM, relational queries, and other sparse matrix kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads while maintaining comparable performance on non-skewed ones. These gains, along with improved bandwidth utilization, remain valid across a wide range of workloads and different hardware systems.

PVLDB Reference Format:
Wentao Huang, Qiming Huang, Yunhong Ji, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

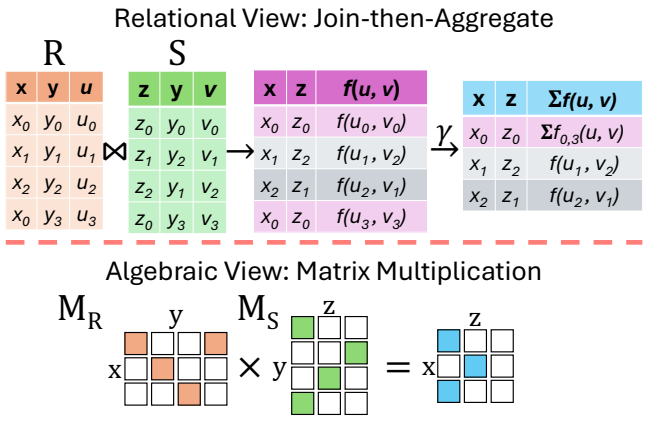

Figure 1: Comparing two approaches for evaluating a join-aggregate query, " *SELECT x, z, SUM(f(u,v)) FROM R, S WHERE R.y = S.y GROUP BY x, z* ", where f denotes a mathematical function. The "relational view" (top) joins tables $R(x, y, u)$ and $S(z, y, v)$ on attribute y followed by an aggregation, a process that produces excessive intermediate results, such as $(x_0, z_0, f(u_0, v_0))$ and $(x_0, z_0, f(u_3, v_3))$, which are ultimately aggregated into one final group-by result (shaded in light pink). The "algebraic view" (bottom) models the operation as a matrix product $M_R \times M_S$ where y serves as the shared dimension to compute the aggregate result directly, bypassing the generation of costly intermediate results.

The source code, data, and/or other artifacts have been made available at https://github.com/fukien/casmm.

1 INTRODUCTION

Sparse general matrix-matrix multiplication (SpGEMM) is a fundamental kernel underpinning numerous applications, including scientific computing, graph analysis, and machine learning [32, 48, 59, 66]. Recently, SpGEMM has emerged as a transformative method for processing relational join-aggregate and project queries [28, 42,

43, 46, 77–79] [1] . By treating relational tables as algebraic sparse matrices, entire query pipelines can be executed as unified algebraic operations. This enables aggregation or projection to be performed *on-the-fly* alongside the join, effectively bypassing the prohibitive costs of materializing excessive intermediate results (see Figure 1 for a comparison) . In all such cases, the performance of the application is dictated by the efficiency of the underlying parallel SpGEMM execution. Reflecting its critical role, numerous studies have investigated SpGEMM optimization, reaching a broad consensus that it is a fundamentally bandwidth-intensive kernel where throughput is ultimately governed by available memory bandwidth [38, 64].

Though it is bandwidth-bound, running SpGEMM on a high-bandwidth platform does not necessarily translate to higher throughput [2] . This is primarily attributed to the *load imbalance* caused by the widely observed skew patterns in real-world workloads [12, 24, 51, 56, 71, 85, 96]. For instance, in a typical OLAP ClickHouse benchmark [73], the top 5% of users generate 24.4% (47.8 million) of all clicks, while the median number of clicks per user is only 2. Similarly, in a common PageRank workload [90], the top 1.8% of pages receive more incoming links than the remaining 98.2% combined. From a matrix perspective, this highly skewed pattern leads to a power-law distribution of the nonzero structure, resulting in significant load imbalance. When running SpGEMM on these workloads, a small subset of cores becomes overloaded while others remain underutilized [32, 38, 64]. This not only limits compute efficiency but also prevents full utilization of the available memory bandwidth, leaving system resources idle, and therefore, rendering high-bandwidth platforms less effective.

Moreover, the challenge of addressing load imbalance in skewed workloads is particularly critical and is further exacerbated by the "memory scaling wall", where memory technology advancements lag behind those of processing units [70, 82]. This trend results in decreasing memory bandwidth and capacity per processing core [45, 88], which further compounds the load imbalance problem for two reasons:

First, decreasing bandwidth-per-core has an acutely negative impact on SpGEMM workloads with load imbalance, where a few straggling threads drag down performance. Since these threads largely rely on core-level bandwidth, their overall performance is inevitably reduced due to the insufficient bandwidth-per-core caused by the "memory scaling wall."

Second, the prevalent approach to compensating for decreasing bandwidth-per-core, adopting heterogeneous memory expansion technologies like CXL [44, 55, 88, 89], can inadvertently amplify this problem. Since CXL-attached memory typically exhibits higher access latency than local DRAM [3] , straggling threads that run disproportionately long are more exposed to CXL's latency penalty, further widening the gap between fast and slow threads. Figure 2 illustrates this using a hashing-based SpGEMM algorithm [64] on a skewed workload, where per-thread runtimes vary due to load imbalance. Because total runtime is governed by the slowest thread, the disproportionate latency on stragglers negates the aggregate bandwidth gain, yielding worse overall performance than pure DRAM. This counterintuitive result is also reported in existing CXL studies [60, 61, 80, 81]. As a result, addressing load imbalance is becoming more imperative than ever.

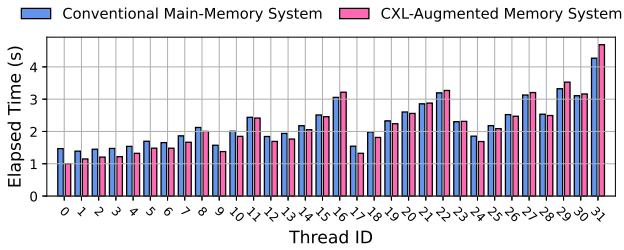

Figure 2: Comparison of per-thread elapsed time for hashing-based SpGEMM [64] on a G500 skewed matrix (scale=17, edgefactor=16) [20] between a conventional main-memory system (28.50 GB/s bandwidth) and a CXL-augmented system (46.50 GB/s bandwidth) (see § 4.1 for experimental setups).

However, addressing load imbalance in SpGEMM is not trivial. While numerous strategies have been proposed [32, 38, 67], we maintain that existing approaches fall short due to two co-existing root causes: *multi-dimensional skew* and *cache contention*, both of which not only hinder load balancing but also degrade overall performance.

First, existing methods fail to accommodate the *multi-dimensional skew* inherent in sparse matrices. Current methods rely on static schemes that balance along a single metric (e.g., flop count) to minimize scheduling overhead. However, skew manifests across multiple dimensions, including computation cost, materialization overhead, and data locality, making it impossible for any single static metric to achieve balanced execution across all threads.

Second, in skewed workloads, existing approaches suffer from significant cache contention, induced by dense substructures that are co-scheduled together. In SpGEMM, irregular sparsity forces practitioners to cache row-wise intermediates to reduce random access costs. However, dense substructures within skewed matrices incur excessive irregular random entry access, which creates heavy cache pressure and even induces significant cache contention—a factor that has long been overlooked as a key performance bottleneck. This cache contention is particularly pronounced under simultaneous multithreading (SMT), which aims to increase computation parallelism but worsens the interference.

In this paper, we address these challenges through a novel approach, *adaptive morsel scheduling (AMS)*. In contrast to de facto methods, AMS revisits *dynamic scheduling*, a strategy conducive to load balancing that is often dismissed due to concerns regarding scheduling overhead and poor performance [38, 64]. Through analysis (§ 2.2), we find that task granularity is the primary performance factor: a granularity that is too fine fails to preserve spatial locality and therefore suffers from excessive cache thrashing. Hence, we employ a morsel-wise dynamic scheduling strategy, where a "morsel" is defined as a basic scheduling unit of several consecutive

[1]The paper focuses on join-aggregate queries, as they produce large intermediate cardinalities that stress the SpGEMM kernel; the same framework applies to project queries, which share the same algebraic formulation.
[2]While SpGEMM above a certain density threshold can be compute-limited, the growing compute-bandwidth gap [34, 57, 70, 82] makes compute progressively less likely than bandwidth to be the primary bottleneck.
[3]Since DRAM is the primary technology used in memory manufacturing, we use the terms "DRAM" and "memory" interchangeably throughout this paper.

matrix rows. AMS ensures a morsel is large enough to maintain spatial locality, thereby greatly mitigating cache thrashing, and more importantly achieves load balance on skewed workloads through dynamic scheduling, effectively preventing computational idling and improving memory bandwidth utilization.

To alleviate cache contention, AMS introduces two lightweight and generalizable techniques: *contention-aware load balancing (CALB)*, which mitigates cache interference by adaptively scheduling dense morsels, and *data-conscious simultaneous multithreading activator (SMTact)*, which selectively enables SMT based on workload contention. These optimizations are simple in design, yet effective in practice, and easy to generalize to other sparse matrix kernels (e.g., SpMM and SpMV). Importantly, their performance benefits remain effective across heterogeneous bandwidth-boosted systems (e.g., CXL systems), ensuring that latency heterogeneity does not cause performance degradation.

Furthermore, the morsel-wise scheduling strategy enables a *hybrid accumulation mechanism (HYB)* for SpGEMM, where different accumulation algorithms can be selectively applied to morsels based on their intrinsic characteristics, thereby achieving more efficient execution and enhanced overall SpGEMM performance.

The contributions of this paper are summarized as follows:

(1) We examine and characterize two co-existing root causes of performance degradation in skewed SpGEMM workloads: *multi-dimensional skew*, where no single static metric can balance all threads, and *cache contention*, where dense substructures induce severe on-chip interference that existing methods overlook.
(2) We develop a novel morsel-wise dynamic scheduling framework, adaptive morsel scheduling (AMS), that systematically addresses both *multi-dimensional skew* and *cache contention* in SpGEMM. AMS decouples skew dimensions through fixed-result-size morsels, integrates two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), to alleviate on-chip cache interference, and enables hybrid accumulation mechanism (HYB) for morsel-adaptive accumulation. Our approach is straightforward in design, yet novel and generalizable, and effective in practice.
(3) We evaluate AMS across a wide range of workloads and show that it maintains on-par performance on non-skewed workloads while significantly outperforming state-of-the-art methods on skewed workloads, addressing computational idling and bandwidth underuse.

The rest of the paper is organized as follows. We first present the background and motivation in § 2, followed by a description of AMS in § 3. We then show the experimental results in § 4 and discuss related work in § 5. Finally, we conclude the paper in § 6.

2 BACKGROUND AND MOTIVATION

We first review state-of-the-art SpGEMM algorithms, then analyze how multi-dimensional skew and cache contention degrade performance under skewed workloads, and finally revisit dynamic scheduling as a promising solution, motivating our AMS approach.

Algorithm 1 Parallel Gustavson's Algorithm for SpGEMM

Input: CSR matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$
Output: CSR matrix $C \in \mathbb{R}^{m \times n}$
for parallel for $i \leftarrow 0$ to $m-1$ **do** ▷ Parallel over rows of A
    Initialize empty `accumulator` for row i
    for all $(k, A_{ik}) \in$ row i of A **do**
        for all $(j, B_{kj}) \in$ row k of B **do**
            if $j \in$ `accumulator` **then**
                `accumulator[`j`]` += $A_{ik} \cdot B_{kj}$
            else
                `accumulator[`j`]` $\leftarrow A_{ik} \cdot B_{kj}$
            end if
        end for
    end for
    Store `accumulator` as row i of C
end for

2.1 The SpGEMM

SpGEMM computes the product $C = AB$, where A, B, and C are all sparse matrices [4] . This form has proven effective across diverse large-scale workloads, including scientific computing, graph analytics, and more recently relational query processing [42, 63, 64, 78]. To reduce memory footprint at scale, sparse matrix formats are widely used, with compressed sparse row (CSR) being the most common representation in existing works [18, 36, 47]. However, the irregular and input-dependent sparsity patterns of sparse matrices, combined with irregular memory access patterns, make SpGEMM a fundamentally bandwidth-intensive workload, where performance depends critically on how effectively memory bandwidth is sustained across cores. Moreover, predicting the number and locations of nonzeros in the output matrix is often intricate, making memory pre-allocation challenging.

To address this pre-allocation challenge, SpGEMM is typically divided into two phases: a symbolic phase, which determines the nonzero structure of the output, and a numeric phase, which performs the actual computation based on that structure. In addition, the irregular nature of sparsity makes it difficult to efficiently gather and accumulate scattered contributions to each output entry. To tackle this, prior work has introduced a variety of accumulator designs to aggregate partial results during computation, including sorted variants such as heaps [63, 64], SPA [35, 40], and expand-sort-compress (ESC) [11, 26], as well as unsorted variants based on hash tables [41, 63, 64].

Building on these data representations, SpGEMM algorithms are commonly implemented using one of three formulations: inner-product [69], outer-product [1, 38], or row-row paradigm (Gustavson's algorithm) [40]. Among these, Gustavson's algorithm operates in a row-wise manner, allowing each row of the left-hand side matrix (A) to be computed independently, enabling straightforward thread-level parallelization without the need for inter-thread synchronization (see Algorithm 1). It is this high degree of parallelism, combined with its efficient memory access patterns, that has

[4]Throughout this paper, we refer to A and B in $C = AB$ as the left-hand side and right-hand side matrices, respectively.

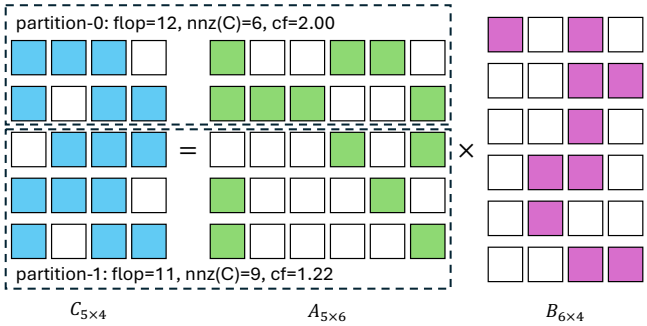

Figure 3: An illustration of $C = AB$ in SpGEMM (colored cells denote nonzero elements), where the two dashed boxes represent partitions with comparable flop counts but differing numbers of output nonzeros and compression factors.

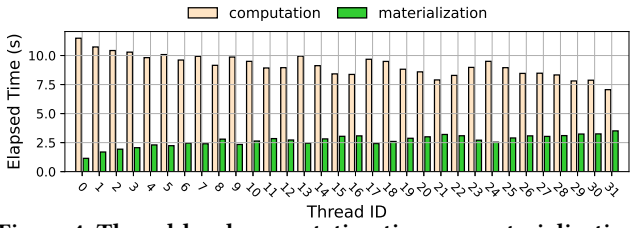

Figure 4: Thread-level computation time vs. materialization time for hashing-based SpGEMM on a G500 skewed matrix (scale=19, edgefactor=8).

made Gustavson's algorithm a dominant formulation for SpGEMM across a wide range of computing platforms.

Following Gustavson's algorithm, several load-balancing techniques have been proposed [1, 3, 9, 16, 22, 84], with static scheduling based on a flop-count scheme emerging as the dominant strategy [63, 64]. This method partitions the rows of matrix A into contiguous regions, each assigned approximately the same number of floating-point operations (i.e., additions plus multiplications). To achieve this, a lightweight preprocessing pass scans matrices A and B to estimate the flop count for each output row of C without performing actual multiplications. Based on these estimates, rows are grouped into partitions such that each thread receives a workload with a nearly equal number of floating-point operations. This approach enables static, flop-count-balanced scheduling across threads while avoiding runtime synchronization.

2.2 The Flop-Count-Based Scheme Under Skew

Despite its simplicity and widespread adoption [18, 64], the flop-count-based scheme is fundamentally limited and fails to achieve effective workload balance under skewed patterns. We identify two root causes, an inherent ignoring of multi-dimensional skew and the performance penalty of overlooking cache contention, both of which hurt load balancing and degrade bandwidth utilization.

(1) Ignoring multi-dimensional skew. Skewed SpGEMM workloads can exhibit imbalance across multiple dimensions, such as submatrix density and row-wise nonzero (*nnz*) distribution. However, the flop-count-based scheme equalizes only the total number of floating-point operations, ignoring other forms of skew. As a result, one thread may receive a few dense rows (with high *nnz*), while another is assigned many sparse rows. Although these assignments may have similar total flop counts, their structural differences lead to divergent runtime behaviors. Dense rows, in particular, induce "clustering" [5] effects during processing, which manifest as temporal bursts of memory and cache pressure. These effects significantly degrade performance, and the extent of impact varies by accumulation strategy [26, 35, 63]. Therefore, the scheme is inherently unable to preserve these skew dimensions.

[5]We use the term "clustering" to describe this effect, following the notion of "primary clustering" in open-addressing hashing.

Additionally, thread-level partitions often exhibit different compression factors (cf) [6] , which leads to divergent trends in computation and materialization times, making it challenging to effectively balance these two workload components. Consider Figure 3: using Gustavson's algorithm, partition-0 (the upper two rows of A) requires 12 flop counts [7] , producing 6 nonzeros; partition-1 (the bottom three rows) requires 11 flop counts but produces 9 nonzeros. Despite both involving comparable amounts of computation, the second has larger output size, inducing more materialization cost.

Empirical data on a skewed workload confirms this disparity, as Figure 4 shows: materialization time varies across threads, with computation time also exhibiting thread-level variation, but in the opposite trend. Moreover, these complexities are further compounded by hardware variability: performance sensitivity to the relative costs of computation versus memory access differs considerably across hardware platforms. This architectural variance makes it exceedingly difficult for purely flop-count-based schemes to establish a load balance that is both universally effective, and readily generalizable across diverse machines. Thus, such schemes are inherently limited in delivering robust load balancing.

(2) Overlooking cache contention. Existing methods also overlook the performance impact of cache contention during parallel execution. In Gustavson's algorithm, each thread maintains an exclusive accumulator whose size scales with the nonzero density of its assigned rows. For high-cf partitions, these accumulators impose significantly greater pressure on shared caches than those of low-cf partitions, making them more prone to inter-thread cache interference. Figure 5(a) shows this effect using the same skewed matrix as in Figure 4. When all threads compute their assigned default partitions, the runtime of a high-cf partition (originally assigned to thread-0 in the flop-count-based scheme) increases substantially: from 2.76 seconds in single-threaded execution (cf. Figure 5(b)) to 8.93 seconds under default multithreaded partitioning, and further to 13.01 seconds if all threads execute this high-cf partition. Note that no sharing benefit arises in this case, as each thread operates on its own exclusive accumulator; instead, these accumulators compete for the same shared cache, amplifying interference as more threads process high-cf regions. In contrast, a low-cf partition (originally assigned to thread-31) sees only a modest runtime increase, from 5.36 to 6.73 seconds.

[6]The compression factor (cf) is defined as the number of flop counts divided by the *nnz* of the result matrix C, i.e., $cf = \frac{flop}{nnz(C)}$ [38].

[7]Only multiplications are counted, as the number of additions is identical.

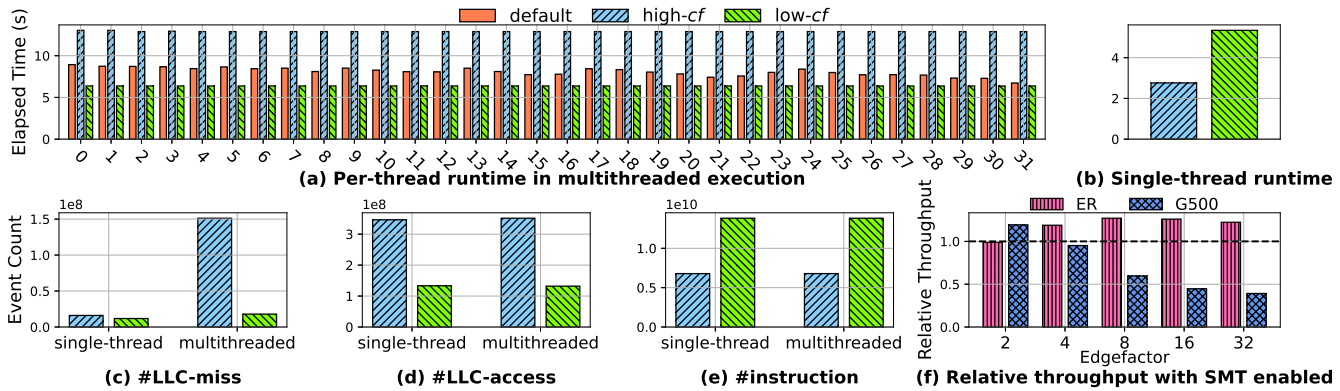

Figure 5: (a) Per-thread computation time of HASH-based SpGEMM on a G500 skewed matrix (scale=19, edgefactor=8) under three settings: all threads execute their default partitions; all execute a high-cf partition (thread-0's); all execute a low-cf partition (thread-31's). (b) Computation time of the low-cf and high-cf partitions under single-threaded execution. (c)–(e) LLC misses, LLC accesses, and instruction counts for the low-cf and high-cf partitions under both single-threaded and multithreaded execution. (f) Normalized throughput of HASH-based SpGEMM with SMT enabled on ER and G500 matrices (scale=18) across varying edgefactors. The black dashed line marks the relative throughput without SMT.

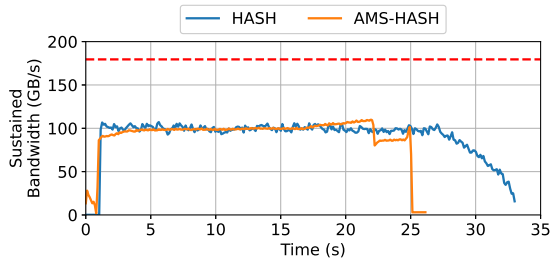

Figure 6: Comparison of sustained bandwidth of the numeric phase between flop-count-based hashing-based SpGEMM and the AMS variant on a G500 skewed workload (scale=19, edgefactor=16); the horizontal red dashed line denotes system memory bandwidth (179.50 GB/s).

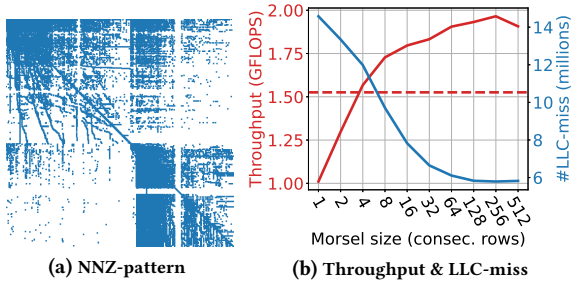

Figure 7: (a) Nonzero pattern of the `webbase-1M` matrix. (b) Throughput and LLC misses of hashing-based SpGEMM on `webbase-1M` using dynamic scheduling with varying morsel sizes (measured in consecutive rows per morsel); the horizontal red dashed line indicates the throughput of the flop-count-based scheme.

Further profiling reveals that cache contention is the root cause of this disparity: Figures 5(c)-(e) show a sharp rise in last-level cache (LLC) misses for the high-cf region, while instruction count and LLC accesses remain relatively stable, confirming cache contention as the primary bottleneck [8] .

The problem is further exacerbated under simultaneous multithreading (SMT): Figure 5(f) shows that non-skewed workloads (e.g., ER matrices with typically low cf) benefit from SMT, whereas skewed workloads (e.g., G500 matrices with high-cf subregions) suffer increasing throughput degradation as density grows, since higher density amplifies the skew and thus intensifies SMT-induced contention. This underscores the importance of contention-aware scheduling; neglecting this factor not only compromises load balance but also degrades overall SpGEMM efficiency.

Ignoring multi-dimensional skew and overlooking cache contention together compound into a macro-level penalty on system-wide bandwidth utilization. As Figure 6 illustrates on a skewed workload, bandwidth resources are only fully utilized during the initial burst of parallel execution; as straggler cores emerge, system-wide bandwidth consumption starts to drop, leaving expensive memory resources idle. This renders prevailing bandwidth expansion technologies, such as CXL, far less effective, as the slowest thread governs total runtime, and its prolonged latency [80] further widens the gap despite the increased aggregate bandwidth available. In contrast, a contention-aware load-balancing approach (our AMS method) sustains markedly higher bandwidth execution.

2.3 Revisiting Dynamic Scheduling

Having established that both multi-dimensional skew and cache contention undermine existing scheduling strategies, we now revisit the scheduling paradigm that enables our solution. Specifically, we revisit *dynamic scheduling* [64, 67]: rows of A are placed into a shared task pool, and threads repeatedly fetch and process one row at a time, achieving nearly 100% load balance. While simple

[8]Write-out time is excluded to directly address computation-level contention.

and theoretically well-balanced, this approach has been shown to underperform compared to flop-count-based scheduling and has largely been dismissed in recent literature, which attributes the performance penalty primarily to the synchronization overhead associated with dynamic task dispatch [38, 64]. However, we contend that it is task granularity, rather than synchronization overhead, that plays the dominant role in determining performance. When task size is carefully managed, dynamic scheduling can match, if not outperform, flop-count-based schemes.

Take a real-world SpGEMM workload, `webbase-1M` [90] for example. The matrix exhibits clear spatial locality across rows, particularly within blocks of consecutive rows (Figure 7(a)). We benchmark the performance of hashing-based SpGEMM using dynamic scheduling with varying task granularities (i.e., the number of consecutive rows per task). Figure 7(b) shows that throughput improves with increasing granularity, plateauing at 256 rows, and surpasses its flop-count-based counterpart due to better load balance. Further analysis of LLC miss counts reveals that larger task sizes result in fewer misses, suggesting better preservation of spatial locality. It is worth noting that many real-world matrices exhibit strong spatial locality in their nonzero patterns, and prior works have introduced several offline reordering algorithms to enhance locality for downstream operations [3, 22, 84]. Based on these observations, we maintain that dynamic scheduling remains a practical and efficient strategy as long as spatial locality is effectively preserved. We therefore build on this insight for the design of our AMS approach.

3 METHODOLOGY

This section presents our proposed framework, adaptive morsel scheduling (AMS). We begin with the core concept of morsel-wise dynamic scheduling, our strategy for addressing multi-dimensional skew and achieving efficient load balancing in SpGEMM. Building upon this foundation, we present two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), designed specifically to alleviate on-chip cache contention. Furthermore, we explain how the morsel-wise framework enables our hybrid accumulation mechanism (HYB) approach for optimizing SpGEMM execution. Finally, we discuss the generalization of AMS to other sparse matrix kernels.

We note that AMS targets the numeric phase of SpGEMM, which dominates both computation and memory traffic. While the same ideas can be applied to the symbolic phase, the benefits are less pronounced; for simplicity, we use standard dynamic scheduling with fixed-size morsels for that phase.

Before detailing our techniques, we briefly digress to outline three key design principles that an efficient SpGEMM solution should satisfy, all of which are met by our proposed AMS.

(1) **Simple and lightweight.** SpGEMM has been extensively optimized over decades [32], and any added complexity can degrade its performance. This is especially critical when SpGEMM is repeatedly invoked as a core kernel in real-world systems [18, 47, 59, 64]. Thus, any proposed method must be simple and lightweight to avoid introducing significant execution penalties.
(2) **Non-disruptive to non-skewed workloads.** While skewed patterns are common in real-world workloads, non-skewed distributions (e.g., uniform distribution, symmetric distribution) also arise frequently. A design that benefits skewed cases but penalizes non-skewed ones forces practitioners to make case-dependent trade-offs, increasing system complexity and operational overhead.
(3) **Generalizable across algorithms and platforms.** An effective SpGEMM solution should generalize across algorithmic variants and hardware platforms. In practice, SpGEMM is used in diverse contexts with varying requirements, e.g., numerical computing often demands sorted output [26], while relational and graph queries typically accept unsorted results [3, 22, 36]. Also, it should adapt to different hardware architectures, where sensitivities to compute and memory bottlenecks vary. A practical solution must therefore accommodate both, supporting sorted and unsorted pipelines while maintaining robust performance across platforms.

3.1 Morsel-Wise Dynamic Scheduling

We previously demonstrated that, by using morsels composed of a sufficient number of consecutive rows, dynamic scheduling is able to effectively preserve spatial locality in sparse matrices. This method significantly reduces LLC misses with negligible synchronization overhead (§ 2.3). We therefore adopt this line of design for SpGEMM execution.

While scheduling work in morsels of a fixed row count helps reduce cache thrashing, we argue that this approach leaves room for further improvement, as it does not address the *multi-dimensional skew* identified in § 2.2. Specifically, it has two limitations: (1) Fixed-row-count morsels do not account for differences in flop count or result *nnz*, making it difficult to balance computation and materialization costs. This limitation is particularly pronounced in skewed workloads, where high-cf submatrices incur high computational expense while others involve substantial write-out overhead. Due to such disparities in two dimensions, modeling inter-thread cache contention becomes challenging. (2) This fixed-row strategy also fails to control the size of the resulting output. A single morsel containing a few dense rows, particularly in skewed workloads, can produce more data than the CPU cache can accommodate, making it challenging to construct a cache-aware analysis model [41].

To address these issues, we propose scheduling morsels based on a fixed result size, i.e., a target $nnz(C)$ per morsel. Recall that SpGEMM consists of two phases, where the symbolic phase determines the per-row nonzero structure. We thus leverage this information to form morsels of consecutive rows in an adaptive manner, such that each morsel contains approximately the same number of output nonzeros. By fixing the materialization cost per morsel, this approach normalizes one dimension of skew, allowing us to focus on the remaining computational cost dimension. Additionally, to facilitate on-chip cache modeling, we limit the morsel size to half of the L2 cache capacity [9].

This strategy is simple and efficient, and it forms the foundation for our subsequent designs. More importantly, it introduces no additional algorithmic phase, fully aligning with our design principle of being **simple and lightweight**.

[9]We empirically find that setting the morsel size to $\frac{1}{8}$ and $\frac{1}{2}$ of the L2 cache offers the best trade-off between performance and cache contention.

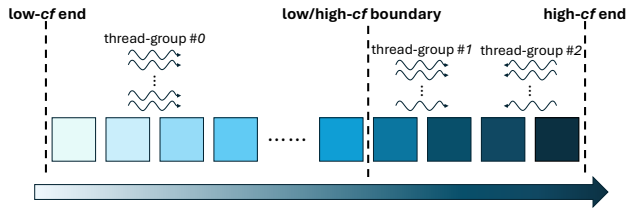

Figure 8: Thread scheduling with contention-aware load balancing (CALB). Darker colors indicate higher compression factors (cf).

3.2 Contention-Aware Load Balancing

We now elaborate on how we alleviate inter-thread on-chip cache contention, which can degrade SpGEMM performance even under balanced workload distribution. Since we form morsels based on a fixed result size, we can easily compute the compression factor (cf) for each morsel. Recall that in skewed workloads, high-cf regions are more prone to contention due to increased cache demands, while low-cf regions are generally more tolerant. Therefore, we aim to co-execute high-cf and low-cf morsels concurrently, alleviating cache contention and reducing susceptibility to inter-thread interference.

To that end, we first sort all morsels in ascending order of cf and empirically set a boundary between low-cf and high-cf regions, typically between the 80th and 95th percentile. This is motivated by the observation that most skewed workloads follow a power-law distribution [3, 22, 84, 96], where a small fraction of dense rows contribute to the majority of the workload. We then divide the available threads into three groups: thread group #0 starts from the low-cf end and progresses toward the high-cf end; group #1 begins at the low/high-cf boundary and proceeds toward the high-cf end; group #2 starts at the high-cf end and moves backward toward the boundary (see Figure 8). The majority of threads are allocated to group #0 to handle low-cf tasks, while smaller subsets are assigned to groups #1 and #2 for high-cf regions. This arrangement ensures that only a limited number of high-cf morsels are processed concurrently. Dividing the remaining threads into groups #1 and #2 further helps mitigate cache contention by avoiding simultaneous execution of multiple contention-prone morsels at the high-cf end. Meanwhile, co-running low-cf morsels, which are less sensitive to cache interference, alongside high-cf morsels helps absorb contention effects and improves overall execution efficiency.

While this scheme alleviates cache contention, it does not guarantee load balance. Some thread groups may finish early and become idle, leading to underutilization of computational and memory bandwidth resources. To address this, we allow early-finished threads to assist other active groups. Specifically, if group #0 reaches the boundary early, it merges with group #1 to continue processing toward the high-cf end. Likewise, early-finished threads from groups #1 and #2 join group #0 to help complete tasks in the low-cf region. This cooperative mechanism ensures that all threads remain active, ensuring efficient use of computation and memory bandwidth.

The CALB scheme is conceptually simple, yet novel and highly effective. To the best of our knowledge, it is the first SpGEMM load-balancing technique that explicitly considers on-chip cache contention as a scheduling factor and thereby delivers tangible performance benefits. Moreover, the method is platform-agnostic and can be seamlessly integrated into any SpGEMM algorithm based on Gustavson's formulation, adhering to the design principle of being **generalizable across algorithms and platforms**.

3.3 Data-Conscious Simultaneous Multithreading Activator

While SpGEMM is generally considered bandwidth-bound, its computational phase still incurs a non-negligible overhead. This overhead can typically be mitigated by leveraging simultaneous multithreading (SMT), which increases the logical core count to effectively improve computational throughput. However, as shown in Figure 5(f), this performance benefit from SMT is primarily observed in non-skewed workloads; in contrast, skewed workloads degrade significantly when SMT is enabled.

We attribute this disparity to the increased on-chip cache pressure from SMT. Although SMT increases the logical thread count, it does not expand the physical cache. Consequently, high-cf morsels suffer from intensified cache contention, an issue that is particularly exacerbated in skewed workloads where cf values are disproportionately large for a few morsels. These specific morsels are highly vulnerable to SMT-induced interference, which in turn leads to substantial performance degradation.

To mitigate this issue while preserving the computational benefits of SMT for non-skewed workloads, we adopt a selective activation strategy of SMT, namely data-conscious simultaneous multithreading activator (SMTact). Specifically, we set a threshold based on the highest cf observed. SMT is disabled if the workload contains any morsels exceeding this threshold, and enabled otherwise. This strategy ensures that SMT is applied only when the workload is less prone to cache contention [10]. The approach is simple yet effective, as it balances the computational benefits of SMT against the risk of elevated cache contention, aligning with our second design principle of being **non-disruptive to non-skewed workloads**.

3.4 Hybrid Accumulation Mechanism

Due to the inherently sparse and irregular nature of SpGEMM, computing a single output row generates multiple intermediate sparse products. These products must then be merged, a process that requires appropriate accumulators to mitigate two challenges: severe irregular memory access patterns and the high cost of managing collisions for identical column indices. Recognizing these challenges, numerous works have focused on developing efficient accumulators [26, 35, 64]. However, the effectiveness of any given accumulation strategy varies dramatically based on the characteristics of the underlying matrix morsel. This makes a static, one-size-fits-all accumulator inherently suboptimal.

To address this, we leverage the morsel-wise framework of AMS to introduce a hybrid accumulation mechanism (HYB) that dynamically selects the most suitable accumulator for each morsel. Specifically, we make this choice based on the aforementioned morsel-wise cf value, selectively choosing between two widely used accumulators: hash tables (HASH) [63, 64], which use linear

[10]We also experiment with using quantiles of the cf distribution instead of the maximum, and observe comparable performance with an appropriately chosen threshold.

probing and multiplicative hashing to reduce collisions, and expand-sort-compress (ESC) [26, 38], which relies on highly optimized radix sorting to alleviate common logarithmic sorting overheads.

The choice between these two accumulators is highly dependent on the morsel-wise cf. As ESC generates sorted output, it executes more instructions than HASH. However, this instruction overhead is not necessarily detrimental, particularly for low-cf morsels. For such morsels, the number of intermediate products is similar to the final number of nonzeros, leading to a moderate cost for merging and sorting. In contrast, HASH becomes more populated for low-cf morsels, and the overhead of hash computation and probing becomes more pronounced. For high-cf morsels, however, HASH is the better choice, as a massive number of intermediates are hashed into a few slots, greatly reducing potential cache misses. Conversely, ESC must pay a significant cache miss cost to merge a large volume of intermediate results into a small final output. Figure 9 corroborates this by microbenchmarking the LLC misses of HASH and ESC, which shows that ESC suffers significantly as the cf increases (e.g., in skewed G500 workloads) compared to non-skewed (ER) workloads.

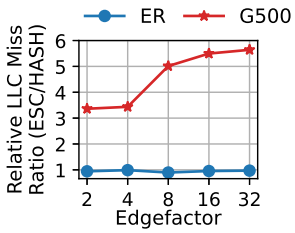

Figure 9: The LLC miss ratio of ESC normalized to the HASH (i.e., LLC Misses of ESC / LLC Misses of HASH), evaluated on ER and G500 datasets at scale 18 with varying edgefactors.

Therefore, we empirically identify a cf threshold, using ESC for morsels with a cf below this value and opting for HASH for morsels above it [11]. This morsel-wise selection also offers an architectural benefit: selecting the accumulator on a per-morsel basis alleviates the instruction-level branch-misprediction costs that finer-grained (per-row) selection would incur.

It is worth noting that by including HASH, our HYB approach may produce an unsorted result matrix, which is acceptable for applications like relational queries and graph analytics that do not require sorted output [46, 63]. However, if sorted output is necessary, HYB can be easily adapted: instead of choosing between HASH and ESC, it can be configured to choose the efficient strategy from a set of sorted accumulators, such as ESC, HEAP [64], SPA [35], or even a hash-then-sort approach [23], to ensure a fully sorted output. Consequently, this adaptive approach ensures each morsel is processed with the most suitable accumulator, leading to better overall SpGEMM performance regardless of the output ordering requirement.

3.5 Extending AMS to Other Sparse Matrix Multiplication Kernels

We now discuss the extension of AMS to other sparse matrix multiplication kernels. Aside from SpGEMM, common sparse matrix kernels include sparse matrix-matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV), both of which multiply a sparse matrix by a dense operand (a matrix or vector). This dense right-hand side makes the output structure (which is also dense) highly predictable, rendering the adaptive accumulation of HYB unnecessary. Similarly, forming morsels based on a fixed result size is not meaningful, as the output size per row is inherently fixed.

However, we contend that AMS's core principles of morsel-wise dynamic scheduling for load balancing and locality preservation are still highly relevant. This relevance stems from the sparse and irregular entries of the left-hand side matrix, which causes row-wise flop counts to vary significantly. As a result, although the multi-dimensional skew present in SpGEMM reduces to a single dimension (flop count) owing to the dense output, the load imbalance and cache contention challenges persist. To resolve this, we can adapt the strategy by forming morsels of consecutive rows based on a targeted flop count (instead of fixed result size). This ensures the direct applicability of AMS's dynamic scheduling strategy while maintaining spatial locality. In addition, since many de facto SpMM and SpMV solutions also follow a Gustavson-style row-wise formulation, AMS's framework is easily adaptable to them.

Furthermore, the strategies for cache-contention alleviation remain applicable. In both SpMM and SpMV, morsels composed of many short rows typically incur more overhead than those of a few long rows [33, 91]. CALB can therefore schedule these morsels of different densities in parallel to strike a global balance of data reuse, as it does for morsels of different cf values in SpGEMM. Additionally, the dense right-hand operand in these kernels yields higher computational throughput (FLOP/s) than SpGEMM typically achieves. Therefore, SMTact can balance inter-thread computational throughput against the data reuse rate in a similar fashion [58].

4 EXPERIMENTAL EVALUATION

This section evaluates AMS in addressing multi-dimensional skew and on-chip cache contention. We first describe the experimental setup, then benchmark AMS on SpGEMM across diverse workloads and dissect CALB and SMTact to validate their contention alleviation. We further evaluate AMS on a bandwidth-expanded CXL system to show improved bandwidth utilization, followed by evaluations on relational join-aggregate queries, an end-to-end Markov clustering pipeline (HipMCL), and other sparse matrix multiplication kernels to demonstrate its broad applicability.

4.1 Experimental Setup

We conduct experiments on a single-socket machine running Linux kernel 6.11.0. This system is equipped with an Intel Xeon® Gold 6430 CPU, which features 32 physical cores and supports 64 logical threads via simultaneous multithreading (SMT) [12] . Each core includes a 48 KB L1 data cache, a 32 KB L1 instruction cache, and a 2 MB L2 cache, while all cores share a 60 MB last-level cache (LLC). The system comprises 192 GB of DDR5 DRAM, providing 179.50 GB/s of memory bandwidth. We also conduct experiments on a CXL-augmented memory system to validate the generalizability of AMS, with the setup details deferred to §4.4.

[11] We find that a cf between 1.5-1.7 provides a robust threshold for selecting the appropriate accumulator.

[12] We disable Intel Speed Select Technology - Base Frequency to ensure equal performance for all physical cores [83].

Table 1: Summary of all evaluated SpGEMM methods

Method	Category	Sorted	Description
Algorithmic baselines (built upon by our AMS methods)			
`HASH [64]`	Baseline	✗	Hashtable accumulator with flop-count-based scheduler.
`ESC [26]`	Baseline	✓	Radix-sort (expand–sort–compress) accumulator with flop-count scheduler.
External / industry competitors			
`HEAP [64]`	Competitor	✓	Heap accumulator with flop-count-based scheduler.
`PB [38]`	Competitor	✓	Outer-product SpGEMM flow with flop-count-based scheduler.
`MKL [47]`	Competitor	✗	Intel oneMKL SpGEMM — non-sorted variant.
`MKLS [47]`	Competitor	✓	Intel oneMKL SpGEMM — sorted variant.
Proposed AMS-based methods			
`AMS-HASH`	Proposed	✗	AMS approach using a HASH accumulator.
`AMS-ESC`	Proposed	✓	AMS approach using an ESC accumulator.
`AMS-HYB`	Proposed	✗	AMS with per-morsel adaptive HASH/ESC selection.

Table 2: Summary of 20 representative matrices evaluated.

Matrix	row(A)	nnz(A)	gini(A)†	flop‡	nnz(A^2)	cf
hugetric-00020	7.13M	21.37M	0.0003	64.08M	44.61M	1.4363
GL7d18	1.96M	35.60M	0.0395	890.29M	843.47M	1.0555
333SP	3.72M	22.22M	0.0559	134.79M	71.97M	1.8730
NLR	4.17M	24.98M	0.0736	152.77M	82.02M	1.8626
M6	3.51M	21.01M	0.0742	128.49M	68.99M	1.8625
AS365	3.80M	22.74M	0.0765	138.91M	74.62M	1.8616
auto	448.70K	6.63M	0.1016	101.25M	33.06M	3.0634
delaunay_n23	8.39M	50.34M	0.1220	316.97M	173.67M	1.8251
nv2	1.46M	37.48M	0.1401	1.28B	292.14M	4.3475
Freescale1	3.43M	17.06M	0.1858	90.33M	38.31M	2.3580
gsm_106857	589.45K	21.76M	0.2240	947.26M	132.40M	7.1546
amazon-2008	735.33K	10.32M	0.2856	218.17M	63.16M	3.4546
webbase-1M	1.01M	3.11M	0.3455	1.37B	1.31B	1.0475
vas_stokes_2M	2.15M	65.13M	0.3771	6.95B	1.17B	5.9452
rel9	9.89M	23.67M	0.4016	2.10B	2.04B	1.0300
vas_stokes_1M	1.10M	34.77M	0.4081	4.39B	721.63M	6.0776
patents	3.78M	14.98M	0.6585	178.30M	132.12M	1.3496
cit-Patents	3.78M	16.52M	0.6844	239.46M	162.35M	1.4749
NotreDame_www	325.73K	929.85K	0.7825	418.01M	288.98M	1.4465
web-NotreDame	325.73K	1.50M	0.8523	503.57M	293.20M	1.7175

† Gini coefficient [54] of the row-wise nonzeros in A.
‡ Only multiplications are counted, as additions are equal in number.

Following prior settings [4, 16, 18, 19, 63], we conduct our primary performance analysis using R-MAT matrix squaring workloads [20, 50]. We employ three representative sets of R-MAT matrices to model different workload characteristics: ER ($a = b = c = d = 0.25$), SSCA ($a = 0.60, b = c = d = 0.13$), and G500 ($a = 0.57, b = c = 0.19, d = 0.05$), which represent non-skewed, skewed, and highly-skewed workloads, respectively. All R-MAT generators support control over size and density via two parameters: *scale* (determining the number of rows) and *edgefactor* (defining the average nonzeros per row). For G500 workloads, higher *edgefactors* amplify multi-dimensional skew and cache contention (§ 2.2). Beyond SpGEMM, we evaluate AMS on relational join-aggregate queries, with setup and workload detailed in § 4.5. Additionally, we evaluate SpGEMM performance using real-world workloads from the SuiteSparse Matrix Benchmark [27]. For a clear presentation, we present results for 20 representative matrices (Table 2), ordered by their Gini coefficient [13], as other matrices show similar trends [14].

We integrate our AMS framework into two prevalent Gustavson's-based SpGEMM algorithms: the non-sorted HASH algorithm [63, 64] and the sorted ESC algorithm [11, 26]. These are referred to as AMS-HASH and AMS-ESC, respectively [15] . Moreover, we implement our hybrid accumulation mechanism (HYB) as AMS-HYB, which dynamically selects between HASH and ESC accumulators. We empirically tune the parameters for AMS-based approaches, configuring morsel size at half the L2 cache size, and distributing threads using a 60%-25%-15% ratio across groups #0, #1, and #2, respectively, for optimal CALB performance. Other parameters are also tuned empirically but are omitted for brevity.

We compare our methods against several state-of-the-art baselines (see Table 1 for an overview). These include: (1) the same HASH and ESC algorithms driven by a state-of-the-art flop-count-based scheduler; (2) the sorted HEAP algorithm [63, 64], which exploits the implicit column ordering of input matrices to reduce sorting overhead at the cost of increased memory usage; (3) the outer-product-based PB method [38]; and (4) the de facto industry solution, Intel MKL [47], including both its sorted (MKLS) and non-sorted (MKL) variants. While other SpGEMM algorithms, such as SPA [35] and SA [41], were also evaluated, we omit their results for conciseness as they consistently underperformed.

All implementations are compiled using `GCC-11.5.0` with the `-O3` optimization flag and parallelized with OpenMP [25]. Baselines are manually tuned for optimal performance, which includes enabling SMT for non-skewed workloads and disabling it for skewed workloads. As in prior works, throughput, measured in GFLOPS (giga floating-point operations per second), serves as the primary evaluation metric. All reported results are averaged over 10 runs .

4.2 SpGEMM Performance Evaluation

Figure 10 presents the SpGEMM performance on R-MAT workloads, and shows that our AMS-based algorithms consistently achieve high performance across this broad spectrum. Specifically, the proposed AMS-HYB is generally the superior solution: it maintains performance comparable to AMS-HASH in skewed workloads (SSCA and G500) while remaining as competitive as AMS-ESC in non-skewed, high-edgefactor workloads (ER). This outcome validates both the effectiveness of our HYB design and the soundness of our cf-based criterion for adaptively selecting the best morsel-wise accumulator for efficient multiplication.

Beyond the superior performance of AMS-HYB, the AMS framework itself significantly boosts both its non-sorted (AMS-HASH) and sorted (AMS-ESC) variants. These methods perform comparably to their respective baselines on ER matrices but deliver a substantial performance advantage on the skewed SSCA and G500 matrices. This demonstrates that AMS's dynamic scheduling, when

[13] The Gini coefficient measures inequality in the row-wise nonzero distribution; a value of 0 indicates uniform distribution, while values approaching 1 indicate extreme skew. Coefficients above 0.30 generally indicate skewed workloads [27, 54].
[14] Our full evaluation covers matrix squaring on 119 SuiteSparse matrices, with row counts ranging from 100K to 10M and nonzero counts from 146K to 316M.
[15] Sorted algorithms generally incur more overhead to produce row-wise sorted output.

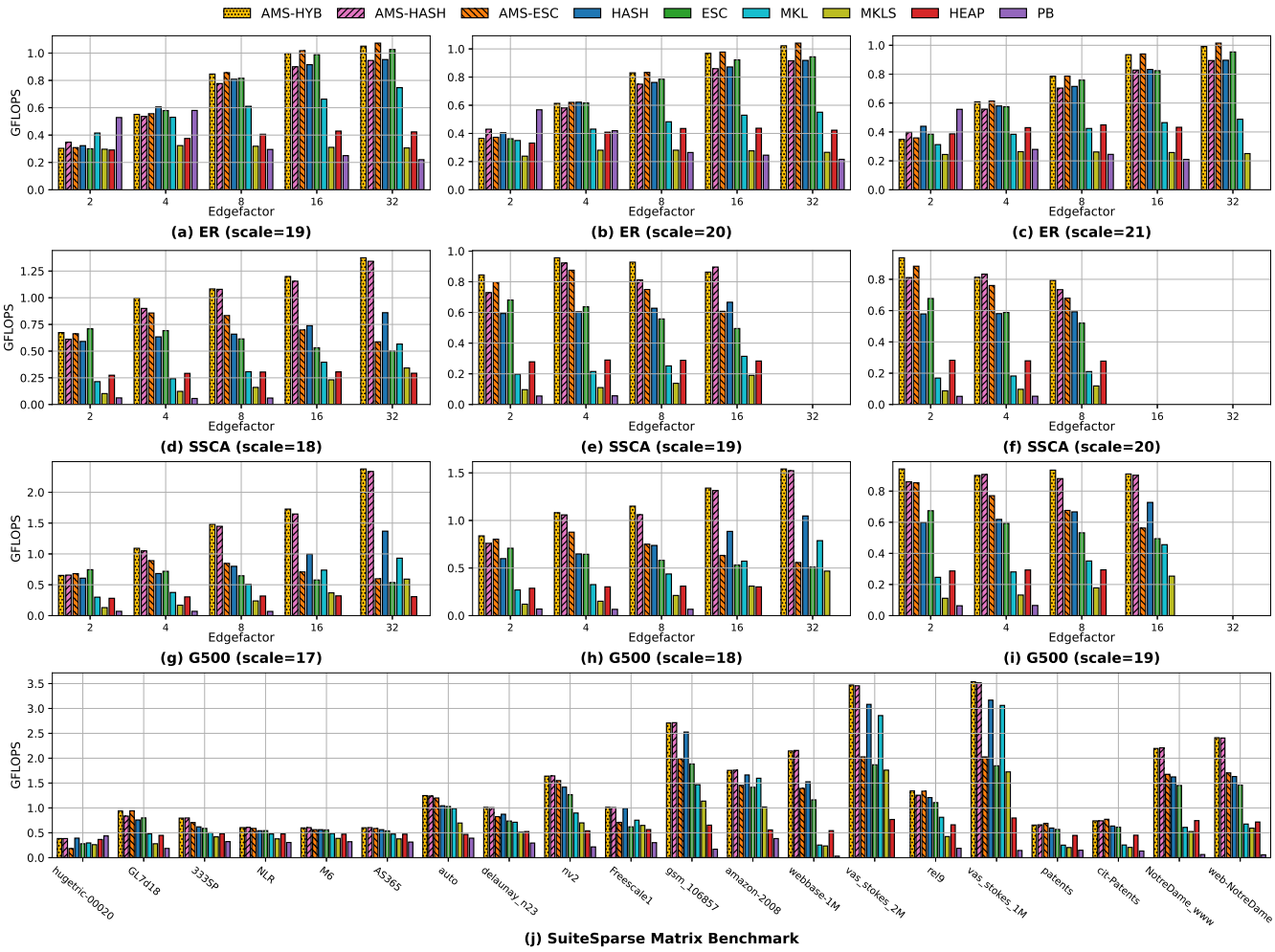

Figure 10: SpGEMM performance evaluation. Subfigures (a)–(i) present results for RMAT matrices (ER, SSCA, G500) across varying scales and edgefactors, representing changes in matrix dimension and density, respectively. Subfigure (j) displays results for 20 representative matrices from the SuiteSparse collection, ordered on the x-axis by their Gini coefficient (ascending), which correlates with increased workload skew. Empty bars signify out-of-memory (OOM) failures.

paired with appropriately sized morsels, effectively addresses multi-dimensional skew on skewed workloads without sacrificing performance on non-skewed ones. The strong results on ER matrices (Figure 10(a)–(c)) also validate the effectiveness of SMTact, which selectively enables SMT in low-contention scenarios to enhance computational throughput, a critical factor in non-skewed workloads.

Comparing the impact of AMS across different algorithms, we observe that the performance improvement is more pronounced in HASH than in ESC. This is because ESC relies on radix sort, which has a complexity of $O(\omega \cdot n)$ and correlates directly with the flop count. Although HASH-based methods also exhibit $O(n)$ complexity in hash table lookups, their efficiency is more susceptible to "clustering" effects, making flop-count-based models less effective in accurately estimating cost and balancing workloads. In contrast, our AMS approach is agnostic to the underlying SpGEMM algorithm. For both sorted and non-sorted algorithms, AMS achieves performance gains of up to 81.01% and 36.03%, respectively, demonstrating its general applicability across algorithmic variants.

To further illustrate the source of these gains, Figure 11 drills down into the per-thread elapsed time, comparing AMS-based methods against their respective baselines on a representative skewed workload. The baseline HASH and ESC exhibit significant thread-level runtime variance, confirming that flop-count-based scheduling fails to balance the actual workload under skew. In contrast, AMS-based variants achieve substantially more uniform per-thread runtimes, demonstrating that our scheduling framework effectively addresses the load imbalance problem inherent in skewed workloads.

Despite these advantages, Figure 10 shows that AMS-based algorithms may occasionally underperform their flop-count-based

counterparts in highly sparse cases (e.g., G500 with scale 17 and edgefactor 2), primarily due to overhead from cf-based morsel sorting. Nevertheless, this penalty diminishes as matrix density increases, underscoring the lightweight nature of AMS in addressing load imbalance.

Turning to the other methods, the baseline HASH and ESC algorithms generally trail their AMS-enhanced counterparts, followed by MKL/MKLS. In contrast, HEAP and PB consistently show inferior performance with more unfinished runs, owing to their notably larger memory demands. An exception is PB's performance on highly sparse non-skewed workloads (ER matrices, edgefactor 2), due to its outer-product-based formulation that maximizes data reuse [38]. However, this performance advantage quickly diminishes at higher edgefactors, implying limited applicability across diverse workloads. These observations further underscore the need for adaptive SpGEMM processing, suggesting that incorporating other SpGEMM formulations (e.g., outer-product or inner-product) into the morsel-wise scheduling framework could yield further improvements, particularly on highly sparse matrices.

Moreover, the analysis of SuiteSparse matrices (Figure 10(j)) substantiates these performance advantages on real-world workloads, confirming AMS's broad practical applicability.

4.3 Dissecting AMS: CALB and SMTact

4.3.1 Effectiveness of CALB. We now examine the performance benefits of our CALB approach. Designed to alleviate cache contention while maintaining load balance, CALB is evaluated against two alternative scheduling techniques also applicable to our cf-sorted morsels: (1) a bidirectional greedy scheduling scheme (BGS), where threads split into two equal-sized groups processing morsels from opposite ends toward the center, and (2) the classical longest-processing-time-first strategy (LPT) [37], in which all threads start from the high-cf end and progress toward the low-cf end.

Figure 12 reports the throughput of BGS and LPT, normalized to our CALB method, for three AMS-based algorithms on G500 workloads (scale=19). The results show that CALB consistently outperforms both alternatives, achieving 3.26%–16.83% and 8.53%–36.80% higher throughput than BGS and LPT, respectively. While BGS also aims to mitigate contention by avoiding excessive thread concentration on high-cf regions, our three-group division strategy in CALB proves more effective at reducing cache pressure, especially at the high-cf end. In contrast, LPT performs poorly precisely because it schedules all threads to handle the most demanding, high-contention tasks simultaneously, thereby exacerbating on-chip cache contention.

These results validate the rationale behind CALB: mitigating cache contention is critical for effective load balancing. Notably, LPT has been a seminal scheduling strategy for over 50 years [37], and our findings highlight the need to revisit and refine classical techniques with the idea of cache pressure alleviation.

We further compare CALB against two widely-adopted OpenMP load-balancing strategies: dynamic and guided scheduling [15, 68]. For a fair evaluation, we implement dynamic scheduling using an empirically determined optimal morsel size of 256 consecutive rows (dynamic-row). Guided scheduling is tested with a minimum morsel set to either 256 rows (guided-row) or 256 floating-point operations (FLOPs) (guided-flop); both configurations are chosen for the purpose of preserving spatial locality.

As Figure 13 illustrates, CALB achieves higher throughput across all comparisons: 11.87%–48.01% over dynamic-row, 12.89%–47.13% over guided-flop, and a substantial 2.79×–3.59× over guided-row. The guided-row method performs particularly poorly because OpenMP guided scheduling progressively reduces its chunk size, which, similar to LPT [37], degrades spatial locality during later processing stages. This result underscores a critical trade-off: effective parallel scheduling must balance load distribution with the preservation of spatial locality.

The remaining OpenMP methods (dynamic-row and guided-flop) effectively balance skewed workloads and maintain spatial locality, yet still fall short of CALB because they do not account for on-chip cache contention. This confirms that the advantage of CALB stems directly from its contention-aware design, which is essential for effective parallel load balancing.

4.3.2 Effectiveness of SMTact. Our SMTact is designed as an optimal SMT enabling strategy by balancing the trade-off between increased computational capability and elevated cache pressure. To evaluate its effectiveness, we compare SMTact against two fixed configurations: SMT always enabled (w/ SMT) and SMT always disabled (w/o SMT). The results, presented in Figure 14, include both non-skewed and skewed workloads. As illustrated, SMTact consistently delivers the best performance across most test cases, validating its ability to activate SMT judiciously. The only notable exception occurs in matrices with very low edgefactors (e.g., edgefactor = 2), where the additional sorting overhead introduced by CALB becomes relatively significant due to the highly sparse nature of the matrix. Although this introduces a minor penalty in such extreme cases, the overhead is quickly offset as edgefactors or matrix scales increase. Given the wide variability in sparsity patterns in real-world SpGEMM workloads, we argue that SMTact offers practical and robust benefits.

4.4 Evaluation on Bandwidth-Expanded System

We now evaluate AMS on a bandwidth-expanded memory system, where CXL boosts aggregate bandwidth but introduces higher access latency. As discussed in §1, load imbalance can inadvertently amplify CXL's latency penalty on straggler threads, negating the bandwidth gain and making effective load balancing particularly critical. For this evaluation, the machine has the same CPU and OS configuration as our previous platform but features host DRAM of 28.50 GB/s bandwidth. Additionally, a CXL expansion memory of 20.95 GB/s bandwidth is installed, yielding a combined system bandwidth of 46.50 GB/s [45, 55, 89].

We evaluate AMS approaches against their vanilla counterparts using skewed (G500) workloads and report the results in Figure 15. As shown, AMS-based methods generally outperform the other methods (Figure 15(a)–(c)). Furthermore, the profiling results confirm that AMS effectively utilizes the additional CXL-augmented bandwidth: AMS-ESC achieves a throughput of up to 40.60 GB/s, far exceeding the DRAM-only peak (28.50 GB/s) (Figure 15(d)). In contrast, the bandwidth consumption of the vanilla counterparts does not exceed that of the host DRAM, indicating that AMS indeed improves bandwidth utilization by addressing load imbalance.

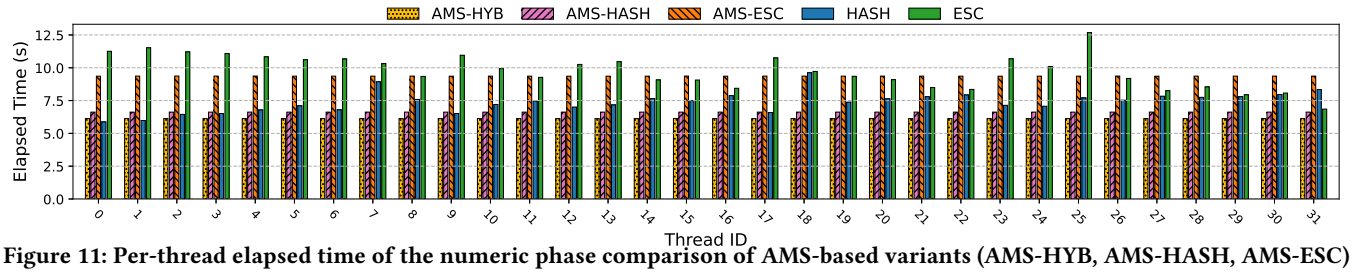

Figure 11: Per-thread elapsed time of the numeric phase comparison of AMS-based variants (AMS-HYB, AMS-HASH, AMS-ESC) and their baseline counterparts (HASH, ESC) on a G500 skewed matrix (scale=19, edgefactor=8).

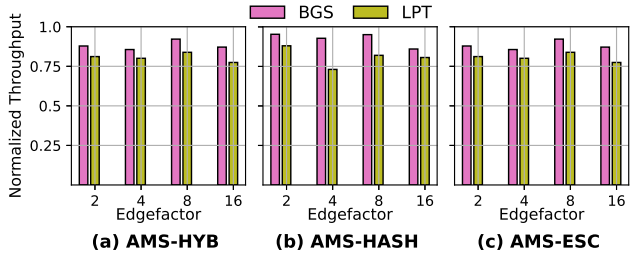

Figure 12: Throughput comparison of BGS and LPT scheduling strategies against CALB. The y-axis shows throughput on G500 (scale=19) workloads, normalized to CALB.

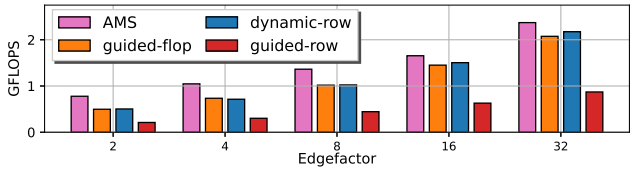

Figure 13: Hashing-based SpGEMM performance on G500 matrices (scale=17) compared to OpenMP dynamic (-row) and guided (-row and -flop) scheduling policies.

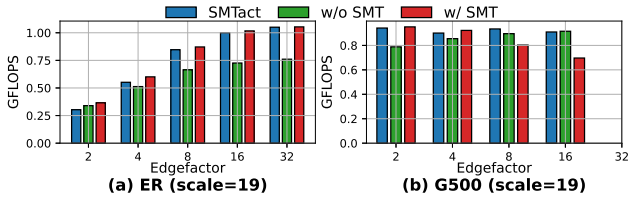

Figure 14: Effectiveness of SMTact in managing SMT to improve AMS-HYB performance on non-skewed (ER) and skewed (G500) workloads.

We also observe an interesting trend in Figure 15(a)–(c): while AMS-based methods remain competitive on skewed workloads, the performance gap between AMS and the vanilla baselines varies between G500 matrices at scale 17 versus those at scales 18 and 19. Taking the HASH-based variants as an example, AMS-HASH outperforms HASH by 58.77% on the G500 matrix at scale 17, delivering 1.53 GFLOPS versus 0.97 GFLOPS (edgefactor 16). However, this advantage diminishes to 23.16% at scale 18, with a similar trend observed at scale 19.

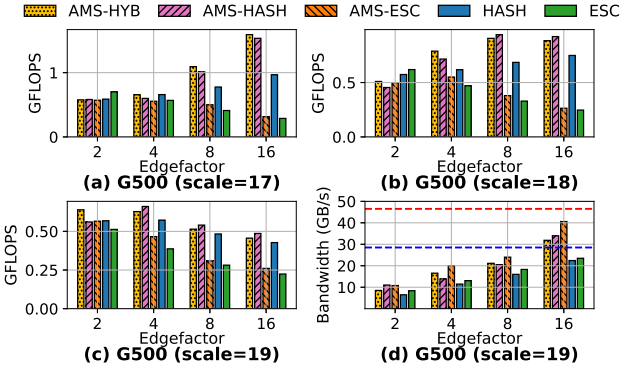

Figure 15: Performance of SpGEMM on G500 workloads. (a)-(c): Measured throughput at G500 scales 17, 18, and 19. (d): Sustained bandwidth measured for different algorithms for the scale-19 workload (the red and blue dashed lines indicate the peak bandwidth of the CXL-augmented memory system (46.50 GB/s) and host DRAM (28.50 GB/s), respectively).

This reduction in the performance gap is mainly attributed to the coincidental load balance occasionally achieved by the static flop-count-based scheme. It is the extent of this imbalance that largely determines the performance degradation of baselines relative to their AMS counterparts. As Figure 4 illustrates, computation and materialization times can exhibit opposing trends across threads. This is precisely the multi-dimensional skew identified in § 2.2: computation and materialization costs trend in opposite directions, and any single-metric scheme can only coincidentally balance them. In some cases, these opposing patterns may randomly offset each other, resulting in a seemingly balanced overall execution time despite the underlying skew. This balance, however, is merely incidental and heavily dependent on the platform's relative sensitivity to computation versus memory access. Consequently, it fails to provide consistent or portable load balance across different architectures.

Previous results on a 179.50 GB/s bandwidth memory corroborate this explanation (cf. Figure 10(g)–(i)). With this higher bandwidth, AMS-HASH achieves a 45.44% performance increase over HASH (1.52 GFLOPS vs. 1.05 GFLOPS) on the G500 matrix at scale 18, because the increased bandwidth significantly reduces the platform's vulnerability to materialization cost. Collectively, these results validate our claim that AMS provides a robust and portable solution to load balancing across diverse systems.

Table 3: Summary of 9 join-aggregate workloads.

Dataset	$\|user\|$	$\|tuple\|$	Gini	$\|OUT_J\|$	$\|OUT_A\|$	$\|OUT_J\|/\|OUT_A\|$
LFSOI [65]	854	458.97K	0.5132	76.67M	728.68K	105.2228
ISF18 [74]	1.82K	1.03M	0.5586	327.58M	3.29M	99.6586
RD23 [76]	6.72K	3.91M	0.5276	4.61B	45.02M	102.3613
BKGN [53]	350.33K	5.15M	0.7242	11.43B	6.77B	1.6892
GDBK [95]	53.42K	5.98M	0.1293	19.63B	2.51B	7.8261
CLHB [73]	850.00K	24.05M	0.3862	20.34B	11.41B	1.7823
JOKE [46]	23.50K	1.71M	0.1709	32.65B	552.25M	59.1219
SAC18 [52]	104.66K	10.00M	0.6988	68.52B	5.94B	11.5325
MVLN [46]	138.49K	20.00M	0.5807	269.61B	16.80B	16.0477

1. $|user|$ and $|tuple|$ are equivalent to the matrix row count and total nnz count, respectively, reflecting the relational-to-sparse-matrix duality; Gini captures the user-wise nnz distribution [54].
2. $|OUT_J|$ and $|OUT_A|$ represent the cardinalities of the join result and the final aggregated result, respectively.

Table 4: Summary of all evaluated join-aggregate query processing approaches

Method	In-place Agg.*	Description
Traditional join followed by aggregation		
`PHJ-AGG [72, 75]`	✗	Radix partitioned hash join with aggregation; deduplication via fast hash table [46, 75].
`NPHJ-AGG [12, 75]`	✓	Non-partitioned hash join; aggregates computed during probing to skip the final deduplication pass [46, 75].
Join-alone algorithms (no aggregation; for comparison only)		
`PHJ [72]`	–	Radix partitioned hash join; serves as the join backend of PHJ-AGG. Does not produce aggregation results.
`NPHJ [12]`	–	Non-partitioned hash join; serves as the join backend of NPHJ-AGG. Does not produce aggregation results.
State-of-the-art join-aggregate techniques		
`DIM3 [46]`	✓	Hybrid sparse/dense join-project query algorithm; intersection-free partitioning eliminates deduplication.
Industry SpGEMM competitors		
`MKL [47]`	✓	Intel oneMKL SpGEMM (non-sorted variant); aggregation replaces deduplication inside the accumulator.
Proposed AMS-based approaches		
`AMS-HYB`	✓	AMS scheduling with hybrid HASH/ESC selection; eliminates the separate deduplication pass.

* *In-place Agg.* denotes whether aggregation is performed on-the-fly alongside the join, or both operations are directly merged into one.

4.5 Join-Aggregate Query Evaluation

We now validate AMS on relational workloads to demonstrate its empirical usefulness. Recent studies have shown that relational tables can be modeled as sparse matrices, and SpGEMM can be used to accelerate join-aggregate or project processing without materializing intermediate results as in the traditional join-then-aggregate paradigm [2, 49], especially for queries involving many-to-many joins or self-joins, which produce large intermediate cardinalities.

Regarding workloads, we employ widely-used datasets from prior works [28, 46, 78], and also evaluate on an industrial OLAP ClickHouse benchmark [73] (see Table 3 for key statistics). These datasets span a wide range of skewness, from low to highly skewed, as quantified by the Gini coefficient in Table 3. As for the query form, we perform self-join (i.e., $R = S$) with aggregation in the form:

```
SELECT R.user, S.user, SUM(R.val * S.val)
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
GROUP BY R.user, S.user
```

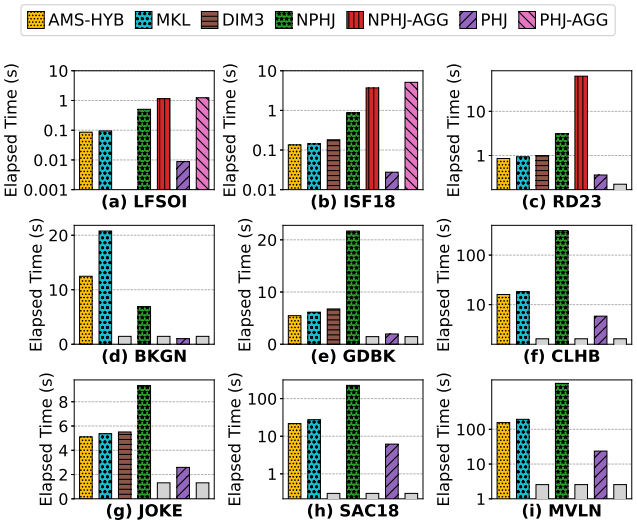

Figure 16: Execution time comparison across diverse join-aggregate workloads; gray bars with "N/A" denote algorithms that failed to complete the execution on those specific workloads; the vertical axes for (d) and (f) are in logarithmic scale to capture the wide range of execution times.

This query measures user-wise similarity, where "item" and "val" abstract the shared attribute and measure across scenarios, e.g., URL/click-count in ClickHouse (CLHB) and movie/rating in MovieLens (MVLN). This form is particularly challenging, as the aggregated values increase the demand for memory, and it is not amenable to existing aggregate query rewriting techniques [21, 29–31, 62, 86, 87, 92, 93].

We compare AMS-HYB with: (1) DIM3 [46], the state-of-the-art join-aggregate method based on SpGEMM; (2) NPHJ-AGG, where a fast hash table [75] performs aggregation on-the-fly alongside the probing phase of non-partitioned hash join (NPHJ) [10, 72], also bypassing intermediate materialization; (3) PHJ-AGG, where we use the cache-efficient partitioned hash join [6–8] to perform the join and materialize the intermediate result, and after that we use the same efficient hash table [75] as NPHJ-AGG to perform the aggregation [16] ; (4) the industrial SpGEMM method: MKL with non-sorted output. For reference, we also include NPHJ and PHJ as join-only baselines (which do not materialize the intermediate results). All evaluated methods are summarized in Table 4 for a clear reference.

Figure 16 presents the join-aggregate results. As shown, AMS-HYB consistently achieves the best performance among all aggregate variants, demonstrating its effectiveness in addressing relational join queries. Surprisingly, even compared with the join-only baseline NPHJ, AMS-HYB wins 8 out of 9 cases, demonstrating the

[16]We do not have PHJ-AGG perform the on-the-fly aggregation as NPHJ-AGG, because on-the-fly aggregation will incur substantial memory traffic to the aggregation hash table, which will break the cache-level efficiency of partitioning, making this benefit completely disappear.

Table 5: Summary of HipMCL [5, 64] Datasets: Viruses and Archaea

dataset	Viruses	Archaea
$\|veres\|$	219,715	1,644,227
$\|edge\|$	9,166,070	409,568,764
Gini	0.6574	0.5759
Iters. to Converge*	37	51

*Number of MCL iterations until the flow matrix converges.

superiority of SpGEMM methods in handling joins with large output cardinalities. Comparing the two runners-up, MKL and DIM3, DIM3 performs slightly worse and cannot finish on 3 workloads due to out-of-memory errors. This is because DIM3 uses an "estimate-partition-multiplication" approach that tends to over-estimate output size with real-world workloads, which are mostly skewed, and thus DIM3 incurs significant memory materialization cost. Notably, the performance advantage of AMS-HYB over MKL is most pronounced on BKGN, the most skewed dataset (Gini = 0.7242), confirming that AMS's ability to address multi-dimensional skew and alleviate on-chip cache contention translates directly to relational workloads where skew is prevalent. Regarding NPHJ-AGG and PHJ-AGG, they finish only on the dataset with the smallest intermediate cardinality. This is because, regardless of whether the aggregation is performed on-the-fly or after the join result is fully populated, the hash table must grow dynamically to hold the unhashed entries that are to be aggregated. This dynamically growing hash table is usually resized with a specific scaling factor (typically 2×), and therefore exhausts the memory footprint very quickly. Beyond memory limitations, comparing the "join-then-aggregate" approaches with their respective join-only baselines, we can see that the aggregation-based methods are an order of magnitude slower, indicating the significant cost of aggregation, and thus underscoring the necessity and importance of SpGEMM-based methods. This is particularly true for PHJ and PHJ-AGG: PHJ alone achieves the best runtime across all evaluated workloads, yet PHJ-AGG yields the worst overall runtime after aggregation (when it can finish the run). This stark contrast indicates that the aggregation operation itself is the dominant bottleneck, underscoring the need for effective join-aggregate query processing.

4.6 Performance Evaluation on Markov Clustering

Beyond SpGEMM kernels and relational join-aggregate workloads, we further evaluate AMS within an end-to-end scientific computing pipeline, namely the HipMCL [5, 64] Markov clustering benchmark. HipMCL identifies clusters in protein-similarity networks by simulating concurrent random walks over the graph, which are realized as repeated sparse matrix squaring of the similarity matrix until a stable clustering structure emerges. Across iterations, the similarity matrix becomes progressively sparser as the clustering structure crystallizes, yet sparse matrix squaring still dominates the overall runtime. We therefore integrate our SpGEMM implementations into HipMCL and report both the SpGEMM kernel time and the total end-to-end execution time on the two HipMCL datasets summarized in Table 5.

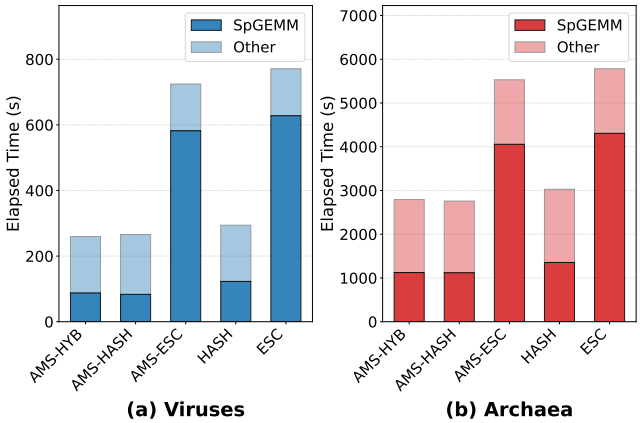

Figure 17: Comparison of HipMCL [5] execution times for (a) Viruses and (b) Archaea datasets. Each bar is partitioned into the time spent on the specific SpGEMM kernel (dark) and the overhead from the remaining HipMCL framework (light).

Figure 17 reports the results. AMS-HYB and AMS-HASH consistently deliver the best performance on both workloads, owing to their effective load balancing and on-chip cache contention alleviation. Compared with the HASH baseline, AMS-HASH reduces SpGEMM kernel runtime by 47.15% on Viruses and 20.91% on Archaea, translating into 10.63% and 9.78% improvements in end-to-end HipMCL execution time, respectively. The larger gain on Viruses can be attributed to two factors: (1) Viruses exhibits a slightly more skewed similarity structure (Gini = 0.6574 vs. 0.5759), which amplifies the benefits of AMS's skew-aware scheduling; and (2) Archaea requires substantially more MCL iterations to converge, and the later iterations operate on increasingly sparse matrices where load imbalance and cache contention become less pronounced, thus diluting AMS's relative advantage. For ESC-based methods, the requirement of producing sorted output incurs noticeably higher SpGEMM execution time overall; nevertheless, AMS-ESC still outperforms the ESC baseline thanks to its superior load balancing and cache contention mitigation. Finally, AMS-HYB closely matches AMS-HASH on both workloads, confirming that HYB reliably selects the appropriate accumulator on a per-morsel basis even under the rapidly evolving sparsity patterns of MCL iterations.

4.7 AMS Evaluation on Other Sparse Matrix Multiplication Kernels

Beyond SpGEMM, join-aggregate queries, and the HipMCL Markov clustering pipeline, we further evaluate the generalizability of AMS on other sparse matrix multiplication kernels, specifically, sparse matrix-matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV).

Since these kernels produce fixed-size dense output, we form morsels by grouping consecutive rows based on a targeted *nnz* count in the left-hand side matrix sub-block (see § 3.5), with each morsel sized at half the L2 cache capacity. The resulting morsels have disparate densities, enabling CALB and SMTact to alleviate cache contention as in SpGEMM.

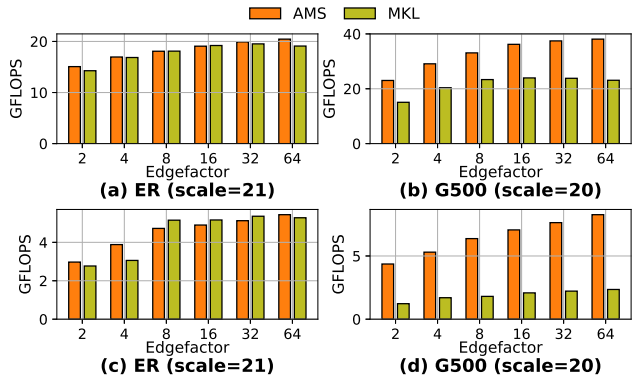

Figure 18: Performance of AMS versus MKL for SpMM (a,b) and SpMV (c,d) operations on ER and G500 workloads.

We compare this AMS-based SpMM and SpMV implementation against the state-of-the-art Intel MKL library on both non-skewed ER and skewed G500 workloads. As shown in Figure 18, our method achieves on-par performance with MKL on non-skewed ER workloads while significantly outperforming it on skewed G500 workloads, with throughput gains of up to 57.25% and 3.51× for SpMM and SpMV, respectively. This demonstrates the broad applicability and robust advantages of AMS's core scheduling principles across various sparse matrix multiplication kernels.

4.8 Discussion

Our experimental analysis across SpGEMM, join-aggregate queries, end-to-end Markov clustering, and other sparse matrix kernels on diverse workloads and hardware platforms reveals several key findings that collectively validate the design rationale and practical effectiveness of AMS.

Multi-dimensional skew is the primary performance bottleneck. Across all SpGEMM evaluations (§ 4.2), we observe that the performance gap between AMS and flop-count-based baselines widens as workload skew increases. On non-skewed ER matrices, AMS performs comparably to the baselines, confirming that it introduces negligible overhead. On skewed G500 matrices, however, AMS delivers substantial gains (up to 81.01% for HASH), demonstrating that multi-dimensional skew, not algorithmic inefficiency, is the dominant factor limiting existing methods. That said, AMS may occasionally underperform in highly sparse cases due to overhead from cf-based morsel sorting; however, this penalty diminishes rapidly as matrix density increases, and is outweighed by the substantial gains on the skewed workloads that dominate real-world applications.

Contention-aware scheduling is essential, not optional. The dissection of CALB and SMTact (§ 4.3) shows that simply achieving load balance is insufficient: methods that balance load but ignore cache contention (e.g., OpenMP dynamic and guided scheduling) still fall short of CALB. Similarly, SMTact's selective SMT activation outperforms both always-on and always-off SMT configurations, confirming that the trade-off between computational throughput and cache pressure must be managed explicitly rather than left to a fixed policy.

Load balancing unlocks hardware potential. The CXL evaluation (§ 4.4) demonstrates that load imbalance can negate the benefits of bandwidth expansion, as straggler threads are disproportionately penalized by CXL's higher access latency. By addressing this imbalance, AMS enables effective utilization of the augmented bandwidth, achieving throughput that exceeds the DRAM-only peak. This finding has broader implications: as heterogeneous memory systems become more prevalent, contention-aware load balancing will be increasingly critical for realizing their performance potential.

SpGEMM-based methods are superior for join-aggregate processing. The join-aggregate evaluation (§ 4.5) reveals that traditional hash-join-based approaches, whether performing aggregation on-the-fly or after materialization, suffer from severe memory scalability issues. In contrast, SpGEMM-based methods bypass intermediate materialization entirely, and AMS-HYB further improves upon the state-of-the-art by addressing skew in relational workloads. The strong results on the highly skewed BKGN dataset confirm that the benefits of contention-aware scheduling transfer directly to relational query processing.

End-to-end gains carry over to graph analytic pipelines. The HipMCL evaluation (§ 4.6) shows that AMS's SpGEMM-level improvements translate into measurable end-to-end speedups in a real-world Markov clustering pipeline, even though the underlying matrices become progressively sparser across iterations. The larger gains observed on the more skewed Viruses dataset are consistent with our earlier findings, reinforcing that the practical value of contention-aware scheduling grows with workload skew.

The framework generalizes beyond SpGEMM. The evaluation on SpMM and SpMV (§ 4.7) confirms that AMS's core scheduling principles—morsel-wise dynamic scheduling, contention-aware load balancing, and selective SMT activation—are not specific to SpGEMM but apply broadly to sparse matrix multiplication kernels that exhibit irregular sparsity patterns.

5 RELATED WORK

SpGEMM has been extensively studied as a critical kernel in scientific computing and graph analytics [26, 66], and has more recently emerged in relational query processing [46, 78]. Over the decades, Gustavson's row-row formulation [40] has become the dominant approach due to its natural parallelizability, particularly when paired with the dense array accumulator known as SPA [35]. However, SPA has been shown to be inefficient and to incur high memory overhead, which prompted the development of more memory-efficient alternatives such as hash tables [64], heaps [63], and radix sort [26].

In addition to algorithmic advances, much attention has been given to the challenges of load balancing in parallel SpGEMM. Schemes based on flop counts [63, 64] have emerged as a dominant solution, along with other static strategies such as methods using blocks [16, 17] or graph partitioning [1]. While effective in certain scenarios, these approaches often fail to account for the multi-dimensional skew characteristics present in real-world SpGEMM workloads, resulting in imbalanced execution.

As a more adaptive alternative, dynamic scheduling offers greater flexibility. For instance, Patwary et al. [67] proposed a dynamic task scheduling scheme that partitions work across both row and

column dimensions. However, task partitioning across two dimensions introduces substantial synchronization overhead, which limits its practical applicability. Similarly, techniques leveraging work-stealing [13, 14, 94] and cache-access modeling [39] have been explored, but either incur significant cache coherence overhead or fail to account for complicated LLC sharing characteristics, making them less favorable than approaches based on flop counts. In another line of work, researchers have adapted dynamic variants of the classical longest-processing-time-first (LPT) algorithm [37], which prioritizes heavy tasks over lighter ones. Critically, none of these approaches account for on-chip cache contention caused by concurrently executing dense submatrices, which AMS explicitly addresses alongside multi-dimensional skew.

6 CONCLUSION

This paper presents adaptive morsel scheduling (AMS), a morsel-wise dynamic scheduling framework that addresses two co-existing challenges in SpGEMM: *multi-dimensional skew* and *on-chip cache contention*. AMS integrates contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact) to alleviate cache contention, and enables hybrid accumulation mechanism (HYB) to adaptively select the best accumulator per morsel. Evaluations across SpGEMM, join-aggregate queries, and other sparse kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads, maintains comparable performance on non-skewed ones, and improves bandwidth utilization, all of which generalize across hardware platforms including bandwidth-expanded systems. Looking ahead, incorporating other SpGEMM formulations (e.g., outer-product or inner-product) into the morsel-wise scheduling framework presents a promising direction for further improvements, particularly on highly sparse matrices.

REFERENCES

[1] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous Input and Output Matrix Partitioning for Outer-Product-Parallel Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 36, 5 (2014).

[2] Rasmus Resen Amossen and Rasmus Pagh. 2009. Faster join-projects and sparse matrix multiplications. In *Proceedings of the 12th International Conference on Database Theory*. 121–126.

[3] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Iwamura. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *IPDPS*. IEEE Computer Society, 22–31.

[4] Ariful Azad, Grey Ballard, Aydin Buluç, James Demmel, Laura Grigori, Oded Schwartz, Sivan Toledo, and Samuel Williams. 2016. Exploiting Multiple Levels of Parallelism in Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 38, 6 (2016).

[5] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic acids research* 46, 6 (2018), e33–e33.

[6] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *Proc. VLDB Endow.* 7, 1 (2013), 85–96.

[7] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *ICDE*. IEEE Computer Society, 362–373.

[8] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2015. Main-Memory Hash Joins on Modern Processor Architectures. *IEEE Trans. Knowl. Data Eng.* 27, 7 (2015), 1754–1766.

[9] Grey Ballard, Alex Druinsky, Nicholas Knight, and Oded Schwartz. 2016. Hypergraph Partitioning for Sparse Matrix-Matrix Multiplication. *ACM Trans. Parallel Comput.* 3, 3 (2016), 18:1–18:34.

[10] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. In *SIGMOD Conference*. ACM, 168–180.

[11] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing fine-grained parallelism in algebraic multigrid methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.

[12] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD Conference*. ACM, 37–48.

[13] Robert D. Blumofe. 1994. Scheduling Multithreaded Computations by Work Stealing. In *FOCS*. IEEE Computer Society, 356–368.

[14] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: An Efficient Multithreaded Runtime System. In *PPoPP*. ACM, 207–216.

[15] OpenMP Architecture Review Board. 1997. OpenMP Fortran Application Program Interface. (1997).

[16] Aydin Buluç and John R. Gilbert. 2008. Challenges and Advances in Parallel Sparse Matrix-Matrix Multiplication. In *ICPP*. IEEE Computer Society, 503–510.

[17] Aydin Buluç and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *IPDPS*. IEEE, 1–11.

[18] Aydin Buluç and John R. Gilbert. 2011. The Combinatorial BLAS: design, implementation, and applications. *Int. J. High Perform. Comput. Appl.* 25, 4 (2011), 496–509.

[19] Aydin Buluç and John R. Gilbert. 2012. Parallel Sparse Matrix-Matrix Multiplication and Indexing: Implementation and Experiments. *SIAM J. Sci. Comput.* 34, 4 (2012).

[20] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *SDM*. SIAM, 442–446.

[21] Surajit Chaudhuri and Kyuseok Shim. 1994. Including Group-By in Query Optimization. In *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo (Eds.). Morgan Kaufmann, 354–366.

[22] YuAng Chen and Yeh-Ching Chung. 2022. Workload Balancing via Graph Reordering on Multicore Systems. *IEEE Trans. Parallel Distributed Syst.* 33, 5 (2022), 1231–1245.

[23] Helin Cheng, Wenxuan Li, Yuechen Lu, and Weifeng Liu. 2023. HASpGEMM: Heterogeneity-Aware Sparse General Matrix-Matrix Multiplication on Modern Asymmetric Multicore Processors. In *ICPP*. ACM, 807–817.

[24] Transaction Processing Performance Council. 2021. TPC BENCHMARKTM DS Standard Specification Revision 3.2.0. https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v3.2.0.pdf

[25] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.

[26] Steven Dalton, Luke Olson, and Nathan Bell. 2015. Optimizing sparse matrix—matrix multiplication for the gpu. *ACM Transactions on Mathematical Software (TOMS)* 41, 4 (2015), 1–20.

[27] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (2011), 1:1–1:25.

[28] Shaleen Deep, Xiao Hu, and Paraschos Koutris. 2020. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1213–1223.

[29] Marius Eich, Pit Fender, and Guido Moerkotte. 2018. Efficient generation of query plans containing group-by, join, and groupjoin. *VLDB J.* 27, 5 (2018), 617–641.

[30] Philipp Fent, Altan Birler, and Thomas Neumann. 2023. Practical planning and execution of groupjoin and nested aggregates. *VLDB J.* 32, 6 (2023), 1165–1190.

[31] Philipp Fent and Thomas Neumann. 2021. A Practical Approach to Groupjoin and Nested Aggregates. *Proc. VLDB Endow.* 14, 11 (2021), 2383–2396.

[32] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-matrix Multiplication. *ACM Comput. Surv.* 55, 12 (2023), 244:1–244:36.

[33] Jianhua Gao, Bingjie Liu, Weixing Ji, and Hua Huang. 2024. A Systematic Literature Survey of Sparse Matrix-Vector Multiplication. *CoRR* abs/2404.06047 (2024).

[34] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39.

[35] John R Gilbert, Cleve Moler, and Robert Schreiber. 1992. Sparse matrices in MATLAB: Design and implementation. *SIAM journal on matrix analysis and applications* 13, 1 (1992), 333–356.

[36] John R. Gilbert, Steven P. Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *PARA (Lecture Notes in Computer Science)*, Vol. 4699. Springer, 260–269.

[37] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal of Applied Mathematics* 17, 2 (1969), 416–429.

[38] Zhixiang Gu, Jose Moreira, David Edelsohn, and Ariful Azad. 2020. Bandwidth Optimized Parallel Algorithms for Sparse Matrix-Matrix Multiplication using Propagation Blocking. In *SPAA*. ACM, 293–303.

[39] Jihu Guo, Rui Xia, Jie Liu, Xiaoxiong Zhu, and Xiang Zhang. 2024. CAMLB-SpMV: An Efficient Cache-Aware Memory Load-Balancing SpMV on CPU. In *ICPP*. ACM, 640–649.

[40] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (1978), 250–269.

[41] Chuhe Hong, Qinglin Wang, Runzhang Mao, Yuechao Liang, Rui Xia, and Jie Liu. 2024. SaSpGEMM: Sorting-Avoiding Sparse General Matrix-Matrix Multiplication on Multi-Core Processors. In *ICPP*. ACM, 1166–1175.

[42] Xiao Hu. 2025. Output-optimal algorithms for join-aggregate queries. *Proceedings of the ACM on Management of Data* 3, 2 (2025), 1–27.

[43] Xiao Hu and Ke Yi. 2020. Parallel algorithms for sparse matrix multiplication and join-aggregate queries. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 411–425.

[44] Wentao Huang, Mian Lu, and Kian-Lee Tan. 2026. Hash Joins Meet CXL: A Fresh Look. In *CIDR*. www.cidrdb.org.

[45] Wentao Huang, Mo Sha, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. 2024. Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing. In *VLDB Workshops*. VLDB.org.

[46] Zichun Huang and Shimin Chen. 2022. Density-optimized intersection-free mapping and matrix multiplication for join-project operations. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2244–2256.

[47] Intel. 2024. Intel® oneAPI Math Kernel Library (oneMKL). https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html

[48] Myung-Hwan Jang, Yunyong Ko, Hyuck-Moo Gwon, Ikhyeon Jo, Yongjun Park, and Sang-Wook Kim. 2023. SAGE: a storage-based approach for scalable and efficient sparse generalized matrix-matrix multiplication. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 923–933.

[49] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2025. Fast Matrix Multiplication meets the Submodular Width. *Proc. ACM Manag. Data* 3, 2 (2025), 98:1–98:26.

[50] Farzad Khorasani, Rajiv Gupta, and Laxmi N. Bhuyan. 2015. Scalable SIMD-Efficient Graph Processing on GPUs. In *Proceedings of the 24th International Conference on Parallel Architectures and Compilation Techniques (PACT '15)*. 39–50.

[51] Changkyu Kim, Eric Sedlar, Jatin Chhugani, Tim Kaldewey, Anthony D. Nguyen, Andrea Di Blas, Victor W. Lee, Nadathur Satish, and Pradeep Dubey. 2009. Sort vs. Hash Revisited: Fast Join Implementation on Modern Multi-Core CPUs. *Proc. VLDB Endow.* 2, 2 (2009), 1378–1389.

[52] Denis Kotkov, Joseph A. Konstan, Qian Zhao, and Jari Veijalainen. 2018. Investigating serendipity in recommender systems based on real user feedback. In *SAC*. ACM, 1341–1350.

[53] Denis Kotkov, Alan Medlar, Alexandr Maslov, Umesh Raj Satyal, Mats Neovius, and Dorota Glowacka. 2022. The Tag Genome Dataset for Books. In *Proceedings of the 2022 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '22)*. 5. https://doi.org/10.1145/3498366.3505833

[54] Jérôme Kunegis and Julia Preusse. 2012. Fairness on the web: alternatives to the power law. In *WebSci*. ACM, 175–184.

[55] Georgiy Lebedev, Hamish Nicholson, Musa Ünal, Sanidhya Kashyap, and Anastasia Ailamaki. 2025. Demystifying CXL Memory Bandwidth Expansion for Analytical Workloads. In *VLDB Workshops*. VLDB.org.

[56] Suhyun Lee, Chaemin Lim, Jinwoo Choi, Heelim Choi, Chan Lee, Yongjun Park, Kwanghyun Park, Hanjun Kim, and Youngsok Kim. 2024. SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs. *Proc. ACM Manag. Data* 2, 6 (2024), 251:1–251:27.

[57] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *Proc. VLDB Endow.* 17, 10 (2024), 2568–2575.

[58] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Yunjun Gao, and Yaofeng Tu. 2025. HC-SpMM: Accelerating Sparse Matrix-Matrix Multiplication for Graphs with Hybrid GPU Cores. In *ICDE*. IEEE, 501–514.

[59] Chunxu Lin, Wensheng Luo, Yixiang Fang, Chenhao Ma, Xilin Liu, and Yuchi Ma. 2024. On Efficient Large Sparse Matrix Chain Multiplication. *Proc. ACM Manag. Data* 2, 3 (2024), 156.

[60] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S. Berger, Marie Nguyen, Xun Jian, Sam H. Noh, and Huaicheng Li. 2025. Systematic CXL Memory Characterization and Performance Analysis at Scale. In *ASPLOS (2)*. ACM, 1203–1217.

[61] Jinshu Liu, Hanchen Xu, Daniel S. Berger, Marcos K. Aguilera, and Huaicheng Li. 2026. Performance Predictability in Heterogeneous Memory. In *ASPLOS (2)*. ACM, 1413–1429.

[62] Guido Moerkotte and Thomas Neumann. 2011. Accelerating Queries with Group-By and Join by Groupjoin. *Proc. VLDB Endow.* 4, 11 (2011), 843–851.

[63] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *ICPP Workshops*. ACM, 34:1–34:10.

[64] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2019. Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors. *Parallel Comput.* 90 (2019).

[65] Tien T. Nguyen, F. Maxwell Harper, Loren Terveen, and Joseph A. Konstan. 2018. User Personality and User Satisfaction with Recommender Systems. *Inf. Syst. Frontiers* 20, 6 (2018), 1173–1189.

[66] Taehyeong Park, Seokwon Kang, Myung-Hwan Jang, Sang-Wook Kim, and Yongjun Park. 2023. Orchestrating Large-Scale SpGEMMs using Dynamic Block Distribution and Data Transfer Minimization on Heterogeneous Systems. In *ICDE*. IEEE, 2456–2459.

[67] Md. Mostofa Ali Patwary, Nadathur Rajagopalan Satish, Narayanan Sundaram, Jongsoo Park, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, Sergey G. Pudov, Vadim O. Pirogov, and Pradeep Dubey. 2015. Parallel Efficient Sparse Matrix-Matrix Multiplication on Multicore Platforms. In *ISC (Lecture Notes in Computer Science)*, Vol. 9137. Springer, 48–57.

[68] Constantine D. Polychronopoulos and David J. Kuck. 1987. Guided Self-Scheduling: A Practical Scheduling Scheme for Parallel Supercomputers. *IEEE Trans. Computers* 36, 12 (1987), 1425–1439.

[69] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. https://doi.org/10.1109/HPCA47549.2020.00015

[70] Moinuddin K. Qureshi. 2014. Memory Scaling is Dead, Long Live Memory Scaling. https://hps.ece.utexas.edu/yale75/qureshi_slides.pdf

[71] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQLStorm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (2025), 4144–4157.

[72] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD Conference*. ACM, 1961–1976.

[73] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.* 17, 12 (2024), 3731–3744.

[74] Mohit Sharma, F. Maxwell Harper, and George Karypis. 2019. Learning from Sets of Items in Recommender Systems. *ACM Trans. Interact. Intell. Syst.* 9, 4 (2019), 19:1–19:26.

[75] Malte Skarupke. 2017. I Wrote The Fastest Hashtable. Probably Dance. https://probablydance.com/2017/02/26/i-wrote-the-fastest-hashtable/ Accessed: 2025-02-21.

[76] Ruixuan Sun, Ruoyan Kong, Qiao Jin, and Joseph A. Konstan. 2023. Less Can Be More: Exploring Population Rating Dispositions with Partitioned Models in Recommender Systems. In *UMAP (Adjunct Publication)*. ACM, 291–295.

[77] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. Accelerating Machine Learning Queries with Linear Algebra Query Processing. In *SSDBM*. ACM, 13:1–13:12.

[78] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. An Empirical Performance Comparison between Matrix Multiplication Join and Hash Join on GPUs. In *ICDEW*. IEEE, 184–190.

[79] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2025. Accelerating machine learning queries with linear algebra query processing. *Distributed Parallel Databases* 43, 1 (2025), 8.

[80] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *MICRO*. ACM, 105–121.

[81] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, Cheng Chen, Hui Zhang, Fei Liu, Shuai Zhang, Xiaoning Ding, and Jianjun Chen. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *EuroSys*. ACM, 818–833.

[82] Micron Technology. 2023. Memory Scaling is Dead, Long Live Memory Scaling. https://sg.micron.com/content/dam/micron/global/public/products/white-paper/cxl-impact-dram-bit-growth-white-paper.pdf

[83] The Linux Kernel Development Community. 2024. *Intel(R) Speed Select Technology User Guide*. https://docs.kernel.org/admin-guide/pm/intel-speed-select.html Linux Kernel Documentation.

[84] James D. Trotter, Sinan Ekmekçibasi, Johannes Langguth, Tugba Torun, Emre Düzakin, Aleksandar Ilic, and Didem Unat. 2023. Bringing Order to Sparsity: A Sparse Matrix Reordering Study on Multicore CPUs. In *SC*. ACM, 31:1–31:13.

[85] Aleksandar Vitorovic, Mohammed Elseidy, and Christoph Koch. 2016. Load balancing and skew resilience for parallel joins. In *ICDE*. IEEE Computer Society, 313–324.

[86] TaiNing Wang and Chee-Yong Chan. 2018. Improving Join Reorderability with Compensation Operators. In *SIGMOD Conference*. ACM, 693–708.

[87] TaiNing Wang and Chee-Yong Chan. 2020. Improved Correlated Sampling for Join Size Estimation. In *ICDE*. IEEE, 325–336.

[88] Marcel Weisgut, Daniel Ritter, Florian Schmeller, Pınar Tözün, and Tilmann Rabl. 2025. CXL-Bench: Benchmarking Shared CXL Memory Access. In *VLDB Workshops*. VLDB.org.

[89] Marcel Weisgut, Daniel Ritter, Pınar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3119–3133.

[90] Samuel Williams. 2008. Williams/webbase-1M. https://sparse.tamu.edu/Williams/webbase-1M

[91] Guoqing Xiao, Chuanghui Yin, Tao Zhou, Xueqi Li, Yuedan Chen, and Kenli Li. 2024. A Survey of Accelerating Parallel Sparse Linear Algebra. *ACM Comput. Surv.* 56, 1 (2024), 21:1–21:38.

[92] Weipeng P. Yan and Per-Åke Larson. 1994. Performing Group-By before Join. In *Proceedings of the Tenth International Conference on Data Engineering, February 14-18, 1994, Houston, Texas, USA*. IEEE Computer Society, 89–100.

[93] Weipeng P. Yan and Per-Åke Larson. 1995. Eager Aggregation and Lazy Aggregation. In *VLDB*. Morgan Kaufmann, 345–357.

[94] Abdurrahman Yaşar, Sivasankaran Rajamanickam, Michael Wolf, Jonathan Berry, and Ümit V Çatalyürek. 2018. Fast triangle counting using cilk. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 1–7.

[95] Mustafa Yazici. 2023. goodbooks 10k the latest. https://www.kaggle.com/datasets/mustafayazici/goodbooks-10k-the-latest. Accessed: 2026-02-21.

[96] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2020. Speeding up SpMV for power-law graph analytics by enhancing locality & vectorization. In *SC*. IEEE/ACM, 86.